Smart Voicemail Router

Roles and Workflow Guide

Document ID: SVR-TEAM-04
Version: 1.0
Updated: 12 November 2026
Companion document: SVR-TEAM-03 Team Allocation v1.3 (the roles and phases in brief)

What this document is

This guide tells every team member exactly what their role means, what to do, and how to do it, from the first clone on their machine to the finished system. Read your own role section fully. Read the parallel workflow and the dependencies section so you know who needs what from you and when. Nothing here is a title. Every role has real work, concrete deliverables, and a clear way to know when it is done.

The project turns a single-speaker Egyptian Arabic customer voicemail into an explainable callback ticket. The dataset is finished. The model selection is settled. What remains is to build the models, integrate the system, evaluate it, and retrain once on the audited labels.

The big picture

The system has four steps, in order:

1. **Audio in.** A voicemail arrives. It is checked, normalized to 16 kHz mono WAV, and transcribed with Whisper large-v3.
2. **Understand.** A model reads the transcript and predicts four fields: complaint category, complaint urgency, customer state, and churn risk. An audio model can add evidence from the voice itself.
3. **Decide.** Triage logic turns the four fields into a queue priority and an explainable callback ticket.
4. **Deliver.** The ticket goes to the agent with the reason, and the whole run is measured for accuracy and latency.

The dataset is the SVR Voicemail Corpus: 6,522 voicemails, 5,751 clean training rows, split train 4,031, validation 1,136, test 584, seed 42. The labels come from a model and are being audited by the team every day for about a month. Training proceeds on the current labels in parallel with the audit. When the audit finishes, the labels are corrected and the models are retrained. That retrain is Phase 4.

The work runs in four phases:

- **Phase 1: Foundations.** Every role builds its own piece on fixtures and mocks. No one waits for anyone.
- **Phase 2: Core models.** The text classifier and the audio model are trained and validated. The winner is picked by validation mean macro-F1.
- **Phase 3: Evaluation and integration.** The test set is evaluated once. The triage logic is finalized on real model outputs. The full pipeline is integrated and the demo is built.
- **Phase 4: Retrain on audited labels.** After the audit finishes, the labels are corrected and the training notebooks are rerun. No architecture or hyperparameter changes.

Getting started: from clone to first run

Every member follows these steps, in order, before starting Phase 1 work. If any step fails, stop and ask Hazem. Do not work around a failed setup.

Step 1: Install the tools

Install Git and Python 3.11 or newer on your machine. Confirm both work:

```
git --version python3 --version
```

Step 2: Clone the repository

```
git clone
https://github.com/hazemelerefey/smart-voicemail-router.git svr cd
svr
```

Step 3: Check out your role branch

Your code lives on your role branch. Check it out:

```
git checkout team/<your-name>-<your-role>
```

Your branch name is in the role table in Section 2 of SVR-TEAM-03 and in your role section below. For example, Sira runs `git checkout team/sira-audio-speech`.

Step 4: Get the data

The dataset lives on the `local-env` branch, separate from the code branches, because the audio is large. Check it out alongside your code with a worktree so you have both at once:

```
git worktree add ../svr-data local-env
```

You now have two folders. `svr/` holds your role branch and your code. `svr-data/` holds the dataset: the audio in `dataset/train`, `dataset/val`, `dataset/test`, the notebooks in `notebooks/`, and the training tables in `final/`. The data folder is your reference. Read from it. Do not edit the audio or the tables in it. Your code points at the data folder with a path, for example `../svr-data/dataset`.

Step 5: Set up the Python environment

Create a virtual environment and activate it:

```
python3 -m venv .venv source .venv/bin/activate
```

Install the packages your role needs. The base packages for everyone are pandas, numpy, and scikit-learn. Each role section below lists the extra packages for that role. Confirm the install works by importing the main package for your role, for example `python -c "import transformers"` for Lama and Sira.

Step 6: Verify the data

Run these checks before you start work. All of them must pass:

1. The training tables exist in `svr-data/final/`: clean_training_train.csv, clean_training_val.csv, clean_training_test.csv.
2. The row counts are train 4,031, validation 1,136, test 584, total 5,751.
3. The audio folders hold the same counts: `dataset/train` 4,031 files, `dataset/val` 1,136 files, `dataset/test` 584 files.
4. Every audio file is a 16 kHz mono WAV and plays.

If any count is wrong, stop and tell Hazem. Do not train on a dataset you have not verified.

Step 7: Start Phase 1

Read your role section below. Do the Phase 1 work first. Push small, reviewable commits to your role branch as you go. Hazem reviews and merges to `develop`.

Role 1: Hazem, Training Lead

Branch: team/hazem-training-lead

What this role means

You own the whole training effort and the final model. You are the person who makes sure the dataset, the models, and the evaluation all fit together, and that the final result is real. You review every member's work and you merge it. Nothing reaches `develop` or `master` without your review.

What you own

- The end-to-end training pipeline, from the clean training tables to the trained model.
- The dataset, including applying the audit corrections when the audit finishes.
- The model selection report and the training decisions that follow from it.
- Review and merge of all six role branches.
- The retrain on audited labels in Phase 4.

What you deliver

- The training notebooks, executed, with outputs saved, on your branch.
- The final trained model and its configuration, with the exact dataset version, seed, and hyperparameters recorded.
- The retrained model after the audit, with the corrections applied and the re-evaluation results.
- A merge record for every accepted change: what it was, what tests or evaluation ran, and what it changed.

Your work, step by step

Phase 1. Set up the training pipeline. Confirm the clean training tables load correctly and the splits match the audio folders. Write the training notebook skeleton that every role will use: data loading, the shared configuration, the seed handling, and the output format for results. Review each member's Phase 1 work as it lands.

Phase 2. Coordinate the model work. Lama runs the text fine-tuning, Sira runs the audio model, Aya runs the evaluation. You review the validation results and confirm the winner by the rule in the model selection report: highest validation mean macro-F1, and if EgyBERT and MARBERTv2 are within one standard deviation, pick MARBERTv2. Record the decision and the numbers behind it.

Phase 3. Oversee the final test evaluation. Confirm the test set is evaluated exactly once, on the frozen 584-row test set, with no re-splitting and no tuning on the test set. Review the error analysis. Confirm the integrated pipeline runs end to end.

Phase 4. After the audit finishes, take the exported audit edits from the audit page. Apply the corrections to the dataset labels. Rerun the training notebooks on the cleaned dataset with the same architecture and hyperparameters. Have Aya re-evaluate. Deliver the final model.

What you need from others

- Aya's evaluation results, to pick the winner and to confirm the final metrics.

- Lama's and Sira's trained models and their validation results.
- Ibthal's triage logic, to confirm the ticket works with the real model outputs.
- Arafa's integrated pipeline, to confirm the system runs end to end.
- The team's audit edits, exported from the audit page, to run Phase 4.

What others need from you

- The training pipeline skeleton and the shared configuration, early in Phase 1, so no one builds on a different setup.
- The dataset, verified and stable, so no one trains on wrong data.
- Reviews and merges, promptly, so no one's work sits unmerged.
- The final decisions on the winner, the split freeze, and the rights and consent questions.

How you know you are done

The training notebooks run end to end without errors. The final model is trained on the corrected labels, evaluated once on the frozen test set, and the metrics are recorded with the dataset version, the model configuration, and the hardware. Every merge is recorded. Nothing is claimed that was not measured.

Role 2: Sira, Audio and Speech Engineer

Branch: team/sira-audio-speech

Extra packages: torch, transformers, librosa, soundfile

What this role means

You own everything that comes from the voice itself: the audio, the transcription, and the audio model. Your job is to prove what the audio can tell us beyond the transcript, and to build the model that extracts it.

What you own

- The audio pipeline: intake checks, normalization to 16 kHz mono WAV, and quality measurement.
- The transcription: Whisper large-v3, its configuration, and the validation that its output is good enough to train on.
- The direct audio classification model: wav2vec2 XLS-R 300M, and the Egyptian checkpoint.
- The audio features that feed the multi-task model, if fusion is used.

What you deliver

- The audio pipeline, with intake checks and normalization, verified on the full dataset.
- The STT validation: a short report showing Whisper large-v3 transcription quality on a sample, with the configuration used.
- The trained audio model, with validation metrics per field.
- The audio features, in a documented format, if Lama uses them for fusion.

Your work, step by step

Phase 1. Verify the audio. Confirm the 5,751 training audio files are 16 kHz mono WAV and match the training tables. Build the intake check that rejects bad audio and records why. Run Whisper

large-v3 on a sample and record the transcription quality and the configuration. Build the XLS-R baseline: load facebook/wav2vec2-xls-r-300m, freeze the encoder, train a simple classifier head on the training split, and record the validation metrics. This baseline tells the team what the audio alone can do.

Phase 2. Train the audio model. Start from the Egyptian checkpoint IbrahimAmin/egyptian-arabic-wav2vec2-xlsr-53 if it loads and validates. Keep the encoder frozen first, then partially fine-tune the top 2 to 4 layers. Record validation metrics per field with the same 5 seeds the text model uses, so the results are comparable. Test audio-text fusion only if the audio baseline shows real signal: if the audio model adds nothing over text alone, say so plainly and do not force fusion. If fusion helps, deliver the audio features to Lama in the documented format.

Phase 3. Support the evaluation. Give Aya the final audio model and its configuration so she can evaluate it on the test set. Help with the error analysis where the audio explains a mistake the text model made.

Phase 4. No new audio work unless the audit corrections change the audio set. If the corrected labels change any audio row, re-run the affected validation and record the change.

What you need from others

- The dataset and the training tables, from Hazem, verified.
- The transcription and the confidence metadata, which are already in the dataset.
- Lama's text model results, to compare audio against text and to decide on fusion.
- Aya's evaluation harness, to run your validation the same way everyone else does.

What others need from you

- The audio pipeline and the intake checks, so bad audio never reaches training silently.
- The STT validation, so the team knows the transcripts are trustworthy.
- The audio model and its validation metrics, so Aya can evaluate it and Lama can decide on fusion.
- The audio features, in the documented format, if fusion is used.

How you know you are done

The audio pipeline verifies all 5,751 training files. The STT validation is recorded. The audio model has validation metrics per field with 5 seeds. The fusion decision is recorded with evidence, whether the answer is yes or no.

Role 3: Lama, Text Classification Engineer

Branch: team/lama-text-classification

Extra packages: torch, transformers, datasets

What this role means

You own the core model: the text classifier that reads a transcript and predicts the four fields. This is the model the whole system depends on. Your job is to train it well, prove it with validation, and hand over a model that is ready to evaluate and integrate.

What you own

- The text classifier: EgyBERT and MARBERTv2 fine-tuning.
- The multi-task setup: one shared encoder with four heads, one per field.

- The fine-tuning configuration: hyperparameters, seeds, and class weights.
- The validation results that decide the winner.

What you deliver

- The fine-tuned text models, with configuration and validation metrics per field.
- The multi-task model, with the shared encoder and four heads.
- The fine-tuning results: validation mean macro-F1 per field, mean and standard deviation across 5 seeds, for both EgyBERT and MARBERTv2.
- The winner recommendation, with the numbers behind it, following the tie-break rule.

Your work, step by step

Phase 1. Set up single-task fine-tuning. Load the clean training tables. Load the tokenizer for EgyBERT and for MARBERTv2. Confirm max sequence length 128 covers the data: the longest transcript is 72 words, about 93 tokens, so 128 is more than enough. Build the training script that fine-tunes one model on one field. Run a short smoke test on a small subset to confirm the pipeline works end to end. Adopt Aya's training infrastructure as soon as it lands: the experiment tracking, the configuration format, and the seed handling. Do not run full fine-tuning on a different setup.

Phase 2. Run the single-task fine-tuning. Train EgyBERT and MARBERTv2 in parallel, one model per field, 5 seeds each (42, 1337, 2023, 7, 99). Use the hyperparameters from the model selection report: learning rate 2e-5, sweep 1e-5, 2e-5, 3e-5; effective batch 16; epochs 3, max 5, early stopping with patience 2 on validation mean macro-F1; warmup ratio 0.1; weight decay 0.01, set explicitly; AdamW; dropout 0.1; gradient clipping max norm 1.0; mixed precision on. Apply the class weights from the dataset, with the churn weight capped at 10 to 20. Record validation mean macro-F1 per field, mean and standard deviation across seeds, for both models. Then build the multi-task model: one shared encoder, four heads, Kendall uncertainty weighting, the saved per-class weights inside each head. Train it the same way and record the same metrics. Compare single-task against multi-task on validation and record which is better per field.

Phase 3. Hand over the winner. Give Aya the winning model and its exact configuration so she can evaluate it once on the test set. Do not tune anything on the test set. Help with the error analysis where the model confuses two categories.

Phase 4. After the audit corrections, rerun the winning configuration on the cleaned dataset. No architecture or hyperparameter changes. Record the new validation and test results.

What you need from others

- The dataset and the training tables, from Hazem, verified.
- The training infrastructure from Aya, early in Phase 1: experiment tracking, configuration format, and seed handling.
- The class weights, which are already in the dataset.
- Sira's audio features, only if fusion is used in Phase 2.

What others need from you

- The fine-tuned models and their validation metrics, so Aya can evaluate the winner and Hazem can confirm the decision.
- The winner recommendation with the numbers behind it.
- The exact configuration of the winning model, so Aya can reproduce the test evaluation and Hazem can rerun it in Phase 4.

How you know you are done

Both models are fine-tuned on all four fields with 5 seeds. Validation mean macro-F1 per field is recorded with mean and standard deviation. The multi-task model is trained and compared. The winner is named with the numbers behind it, following the tie-break rule. The winning configuration is documented and reproducible.

Role 4: Aya, ML Infrastructure and Evaluation Engineer

Branch: team/aya-ml-infrastructure-evaluation

Extra packages: scikit-learn, pandas, numpy

What this role means

You own the machinery that makes the training trustworthy: the infrastructure everyone runs on, and the evaluation that proves whether the models work. You are the person who makes sure a result is real, reproducible, and measured the same way for everyone. If a number cannot be reproduced, it does not count, and you are the one who catches that.

What you own

- The training infrastructure: experiment tracking, hyperparameter sweeps, configuration format, and seed handling.
- The TF-IDF baseline, the reference every model is compared against.
- The evaluation harness: the code that runs a model on a split and reports the metrics.
- The metrics, the error analysis, and the evaluation reports.
- The rule that the test set is evaluated exactly once.

What you deliver

- The training infrastructure, documented, with the configuration format and the seed handling, delivered early in Phase 1.
- The TF-IDF baseline, with validation metrics per field.
- The evaluation harness, with the metrics it reports and the format of its output.
- The evaluation reports: validation results for every model, the final test evaluation, and the error analysis.
- The run records for every experiment: run ID, hypothesis, changed factor, held-constant conditions, measures, and artifacts.

Your work, step by step

Phase 1. Build the infrastructure first, before anyone runs full training. Define the configuration format: model, hyperparameters, seed, dataset version, and split. Define the seed handling: 5 seeds (42, 1337, 2023, 7, 99), every experiment runs all 5 and reports mean and standard deviation. Define the run record format: run ID, hypothesis, single changed factor, held-constant conditions, measures, and artifacts. Build the evaluation harness: it takes a trained model and a split, runs it, and reports per-field macro-F1, accuracy, and the confusion matrix, in a fixed output format. Build it on fixtures first, with mock predictions, so it works before any real model exists. Then run the TF-IDF baseline: TF-IDF features plus logistic regression, one model per field, on the training split, evaluated on the validation split. Record the validation metrics. This baseline is the reference the neural models must beat.

Phase 2. Support the fine-tuning. Help Lama and Sira run their experiments on the infrastructure. Run the hyperparameter sweep for the text model with Lama: learning rate 1e-5, 2e-5, 3e-5, and record the validation result for each. Run the validation for every model and record the metrics the

same way every time. Do not let anyone report a result without a run record.

Phase 3. Run the final test evaluation. Take the winning model from Lama and the audio model from Sira. Evaluate each exactly once on the frozen 584-row test set. Record the per-field metrics, the sample counts, and the failures. Then run the error analysis: where does the model fail, what do the failures have in common, and what would fix them. Write the evaluation report with the actual numbers, the configuration, the hardware, and the limitations. State plainly what the data cannot support.

Phase 4. After the audit corrections, re-evaluate the retrained model on the test set, once, the same way. Record the new results alongside the old ones so the change is visible.

What you need from others

- The dataset and the training tables, from Hazem, verified.
- The trained models from Lama and Sira, with their exact configurations, to evaluate them.
- The triage logic from Ibthal, to evaluate routing agreement.
- The integrated pipeline from Arafa, to measure end-to-end latency.

What others need from you

- The training infrastructure, early in Phase 1, so Lama and Sira run on the same setup.
- The evaluation harness, so everyone measures the same way.
- The TF-IDF baseline, so every model has a reference to beat.
- The evaluation reports, so Hazem can confirm the winner and the team can see the real numbers.
- The run records, so every experiment is traceable.

How you know you are done

The infrastructure is documented and in use by Lama and Sira. The TF-IDF baseline is recorded. The evaluation harness runs on fixtures and on real models. The final test evaluation is recorded with the exact configuration and hardware. The error analysis is written. Every experiment has a run record.

Role 5: Ibthal, Triage and Routing Engineer

Branch: team/ibthal-triage-routing

Extra packages: pandas, numpy

What this role means

You own the decision: how the four model outputs become a queue priority and a callback ticket that an agent can act on. Your job is to make the system's decision explainable. An agent must be able to read the ticket and know why this customer is called first, and what the customer needs.

What you own

- The queue priority logic: how complaint category, complaint urgency, customer state, and churn risk map to a callback priority.
- The callback ticket generation: the ticket format, the fields it carries, and the reason it shows.
- The routing agreement evaluation: how well the system's routing matches a human-reviewed ideal.
- The rule that urgency comes from the complaint only, never from tone, and that state and urgency

are never conflated.

What you deliver

- The triage rules, written down, with the priority each combination of fields produces.
- The ticket format, with every field it carries and what each field means.
- The ticket generator, which takes the four model outputs and produces a ticket.
- The routing evaluation: agreement with the human-reviewed ideal, and the cases where the system disagrees.

Your work, step by step

Phase 1. Write the triage rules. Start from the field definitions: complaint category has 11 values, complaint urgency has 3, customer state has 5, churn risk has 2. Define the priority: high urgency goes first, at-risk churn goes first, and the combination of the two goes first of all. Write the rules as a table, every combination of urgency and churn risk mapped to a priority, so there is no ambiguity. Define the ticket format: customer reference, the four fields, the priority, the route reason in plain words, the suggested opening line, and the contributing rule version. Build the ticket generator on fixtures: feed it mock model outputs and confirm the ticket it produces matches the rules table.

Phase 2. Test the rules against the validation outputs. Take Lama's validation predictions and run them through the ticket generator. Check that the priorities look right and that no ticket is missing a reason. Record the cases where the rules produce a surprising priority and adjust the rules if the definitions support it. Do not change the field definitions; only the mapping from fields to priority.

Phase 3. Finalize on the real model outputs. Take the final test predictions from Aya and run them through the ticket generator. Measure routing agreement against the human-reviewed ideal: the target is 80 percent. Record the agreement and the disagreements, and what each disagreement was. Hand the ticket generator to Arafa for integration.

Phase 4. If the audit corrections change any label, re-run the routing evaluation on the corrected data and record the change.

What you need from others

- The field definitions and the taxonomy, from Hazem, already agreed.
- The model outputs, from Lama and Sira, first as validation predictions in Phase 2, then as test predictions in Phase 3.
- The routing agreement target and the human-reviewed ideal, from the team.
- Aya's evaluation harness, to measure routing agreement the same way everything else is measured.

What others need from you

- The triage rules, written down, so the team can review them before they are implemented.
- The ticket format, so Arafa can build the demo around it and the agent can read it.
- The ticket generator, so Arafa can integrate it into the pipeline.
- The routing evaluation, so the team knows whether the routing works.

How you know you are done

The triage rules are written as a complete table with no missing combination. The ticket generator produces a correct ticket for every mock input. Routing agreement is measured on the real test predictions and recorded with the disagreements. The ticket format is documented and in use in the

demo.

Role 6: Arafa, Systems Integration and Demo Engineer

Branch: team/arafa-systems-demo

Extra packages: fastapi, uvicorn, gradio

What this role means

You own the finished system: the pipeline that runs end to end, and the demo that shows it working. Your job is to take the pieces everyone else builds and make them work together as one system, then prove it runs cleanly and fast enough.

What you own

- The end-to-end integration: voicemail in, ticket out, with every step connected.
- The API and the demo: the interface the team and the competition judges use.
- The latency measurement: p95 under 10 seconds for clips up to 60 seconds.
- The competition deliverables that show the system running.
- The reproducible local run: anyone on the team can run the system the same way.

What you deliver

- The integrated pipeline, with every step connected and the data flowing through it.
- The demo, running cleanly, with upload and browser recording.
- The API, with the endpoints the demo uses.
- The latency report: p95 latency for clips up to 60 seconds, measured separately by CPU, GPU, or provider configuration.
- The run instructions, so anyone can reproduce the demo.

Your work, step by step

Phase 1. Build the skeleton. Define the pipeline stages: intake, STT, classification, triage, ticket. Build each stage as a separate module with a clear input and output. Build the skeleton with mocks first: the STT stage returns a fixed transcript, the classification stage returns fixed fields, the triage stage returns a fixed ticket. Confirm the data flows from the first stage to the last. Build the demo skeleton: an upload, a run, and a ticket display. Keep the API and data contracts stable so the other roles can build against them.

Phase 2. Replace the mocks with real components as they land. When Sira's audio pipeline is ready, plug it in. When Lama's text model is ready, plug it in. When Ibthal's ticket generator is ready, plug it in. Test each replacement in isolation before moving on. Record which components are real and which are still mocks, so the team always knows the state of the pipeline.

Phase 3. Integrate and measure. With all real components in place, run the full pipeline on the validation set and confirm the tickets are correct. Measure the latency: run the pipeline on clips up to 60 seconds and record the p95 latency, separately for CPU and GPU if both are used. The target is p95 under 10 seconds. If the latency is over target, find the slowest stage and report it to the owner of that stage. Build the final demo: upload, browser recording, live queue, ticket evidence, and callback brief. Run the demo five consecutive times cleanly. Write the run instructions so anyone on the team can reproduce it.

Phase 4. After the retrain, update the demo with the retrained model and re-run the latency measurement. Confirm the demo still runs cleanly.

What you need from others

- The audio pipeline from Sira, to plug into the intake stage.
- The text model from Lama, to plug into the classification stage.
- The ticket generator from Ibthal, to plug into the triage stage.
- The evaluation harness from Aya, to measure latency the same way everything else is measured.
- The API and data contracts, agreed with the team, so the components fit together.

What others need from you

- The pipeline skeleton and the API contracts, early in Phase 1, so everyone builds components that fit.
- The integrated pipeline, so the team can see the system working end to end.
- The demo, so the competition deliverables are real.
- The latency report, so the team knows whether the system is fast enough.
- The run instructions, so anyone can reproduce the demo.

How you know you are done

The pipeline runs end to end with all real components. The demo runs cleanly five consecutive times. The latency report shows p95 under 10 seconds for clips up to 60 seconds, with the hardware recorded. The run instructions let anyone on the team reproduce the demo.

How we work in parallel

The phases are designed so that no one waits in Phase 1, and the waiting in later phases is the natural order of the work, not a stall. The table below shows who does what in each phase.

Phase	Hazem	Sira	Lama	Aya	Ibthal	Arafa
Phase 1: Foundations	Training pipeline skeleton, shared config	Audio pipeline, STT validation, XLS-R baseline	Single-task setup, smoke test, adopt Aya's infra	Training infra, eval harness, TF-IDF baseline	Triage rules table, ticket format, generator on fixtures	Pipeline skeleton on mocks, demo skeleton, API contracts
Phase 2: Core models	Coordinate, review validation, confirm winner	Audio model, fusion test	Single-task fine-tuning, multi-task model	Hyperparameter sweep, validation, run records	Test rules on validation outputs	Replace mocks with real components as they land
Phase 3: Evaluation and integration	Oversee test evaluation, review error analysis	Support evaluation, audio error analysis	Hand over winner, support error analysis	Final test evaluation once, error analysis, report	Finalize triage on real outputs, routing agreement	Full integration, latency measurement, final demo

Phase	Hazem	Sira	Lama	Aya	Ibthal	Arafa
Phase 4: Retrain	Apply audit corrections, rerun training	Re-run validation if audio rows changed	Rerun winning config on cleaned data	Re-evaluate retrained model, record change	Re-run routing evaluation on corrected data	Update demo with retrained model, re-measure latency

The label audit runs every day in parallel with all four phases. Every member audits labels on the live audit page daily. The audit does not block training.

Dependencies and waiting points

This section lists every place where one member needs another member's output, and what to do about it. Read it before you start a phase so you know what you are waiting for and what you can do meanwhile.

Waiting point 1: Lama waits for Aya's training infrastructure

Lama's full fine-tuning runs on Aya's infrastructure: experiment tracking, configuration format, and seed handling. Aya delivers the infrastructure early in Phase 1, before the baseline. Lama does not sit idle: Lama builds the single-task setup, the data loading, the tokenizer, and the model loading in Phase 1, and runs a smoke test on a small subset with a minimal configuration. As soon as Aya's infrastructure lands, Lama adopts it and runs the full fine-tuning. If Aya's infrastructure is late, Lama tells Hazem immediately so the plan can adjust.

Waiting point 2: Ibthal waits for real model outputs to finalize triage

Ibthal writes the triage rules and the ticket format in Phase 1 using the field definitions and mock model outputs. No model is needed for that. Ibthal finalizes the rules in Phase 3 against Lama's and Sira's real test predictions. That is the natural order: the rules are written first, then checked against reality. Ibthal does not wait in Phase 1 or Phase 2.

Waiting point 3: Arafa waits for real components to integrate

Arafa builds the pipeline skeleton and the demo skeleton in Phase 1 using mocks. No component is needed for that. Arafa replaces the mocks with real components as they land in Phase 2 and Phase 3. The final integration waits for the final models in Phase 3. That is the natural order: the skeleton is built first, then the real pieces are fitted into it. Arafa does not wait in Phase 1.

Waiting point 4: Aya waits for models to evaluate

Aya builds the evaluation harness in Phase 1 on fixtures with mock predictions. No model is needed for that. Aya runs the TF-IDF baseline in Phase 1, which needs no neural model. Aya evaluates the real models in Phase 3, when Lama and Sira hand them over. That is the natural order: the harness is built first, then the real models are measured with it. Aya does not wait in Phase 1 or Phase 2.

Waiting point 5: Fusion waits for Sira's audio features

If audio-text fusion is pursued in Phase 2, Lama needs Sira's audio features. Sira delivers the features in Phase 1 and Phase 2. Lama's multi-task model starts text-only and adds the audio features when they are ready. Fusion is optional and decided by evidence: if the audio baseline shows no real signal over text alone, there is no fusion and no waiting. No one is blocked by fusion.

Waiting point 6: Phase 4 waits for the audit

The retrain cannot start until the audit finishes, about a month after it starts. This is the one hard wait,

and it is by design. Phases 1, 2, and 3 proceed in parallel with the audit and do not wait for it. The team trains on the current labels, and when the audit finishes, Hazem applies the corrections and the team reruns the training notebooks. No architecture or hyperparameter changes are needed for the retrain, so it is a rerun, not a rebuild.

The rule for waiting

If you are waiting for another member's output, do not stop. Build on fixtures or mocks, write the tests, document the interface, and prepare the integration. When the real output lands, swap it in and test. If the wait is longer than expected, tell Hazem. No one waits silently.

Working rules

These rules apply to everyone, in every phase.

- **Branches.** Work only on your own role branch. Never commit directly to `develop` or `master`. Hazem merges to `develop`, and merges the final validated result to `master`.
- **Commits.** Push small, reviewable commits as you go. Each commit message says what changed and why. A commit that changes the dataset, a schema, or an interface says so in the message.
- **Reviews.** Hazem reviews every push before it merges. A review asks for evidence: what test ran, what the result was, and what changed. A passing test does not by itself prove the change is correct.
- **Experiments.** Every experiment has a run record: run ID, hypothesis, single changed factor, held-constant conditions, measures, and artifacts. Change one experimental factor at a time. Do not infer a winner from a different split or from several simultaneous changes.
- **Test set.** The test set is evaluated exactly once, after validation is frozen. No tuning on the test set. No re-splitting after results exist.
- **Labels.** The labels come from a model and are being audited. Never present a label as human-verified until the audit approves it. The audit page is the only place labels are approved or edited.
- **Meetings.** One weekly integration and decision meeting. Each member reports completed deliverables, evidence and tests, experiment changes and results, blockers, and the next dependency.
- **Secrets.** Never put secrets, passwords, tokens, or private recordings in the repository, in a commit, or in a chat message. Use synthetic fixtures for routine development.
- **Language.** Documents and code comments are in English. Tech terms stay in English.

The finish line

The project is done when all of the following are true:

1. The text model and the audio model are trained, validated, and evaluated once on the frozen test set, with the metrics recorded.
2. The triage logic produces a correct ticket for every test prediction, with routing agreement measured.
3. The pipeline runs end to end with all real components, and the demo runs cleanly five consecutive times.
4. The latency report shows p95 under 10 seconds for clips up to 60 seconds, with the hardware recorded.
5. The audit is finished, the corrections are applied, and the models are retrained and re-evaluated on the corrected labels.
6. Every experiment has a run record. Every merge is recorded. No claim is presented that was not measured.
7. The competition deliverables are complete: the demo, the run instructions, and the results

summary with the actual numbers.

When the finish line is reached, Hazem merges the final validated result to `master`.